Data is often loadable in short depth: Quantum circuits from tensor networks for finance, images, fluids, and proteins

Raghav Jumade[1] and Nicolas PD Sawaya[2,3,*]

[1] *Intel Corporation, Hillsboro, OR 97124, USA*
[2] *Intel Labs, Santa Clara, CA 95054, USA (former affiliation)*
[3] *HPI Biosciences, Oakland, CA 94608, USA; and Azulene Labs Inc., San Francisco, CA 94115, USA (current affiliations)*

Though there has been substantial progress in developing quantum algorithms to study classical datasets, the cost of simply *loading* classical data is an obstacle to quantum advantage. When the amplitude encoding is used, loading an arbitrary classical vector requires up to exponential circuit depths with respect to the number of qubits. Here, we address this "input problem" with two contributions. First, we introduce a circuit compilation method based on tensor network (TN) theory. Our method—AMLET (Automatic Multi-layer Loader Exploiting TNs)—proceeds via careful construction of a specific TN topology and can be tailored to arbitrary circuit depths. Second, we perform numerical experiments on real-world classical data from four distinct areas: finance, images, fluid mechanics, and proteins. To the best of our knowledge, this is the broadest numerical analysis to date of loading classical data into a quantum computer. The required circuit depths are often several orders of magnitude lower than the exponentially-scaling general loading algorithm would require. Besides introducing a more efficient loading algorithm, this work demonstrates that many classical datasets are loadable in depths that are much shorter than previously expected, which has positive implications for speeding up classical workloads on quantum computers.

arXiv:2309.13108v3 [quant-ph] 27 Dec 2023

Introduction. Though the most natural application area for quantum computation is the simulation of quantum physics, there has been substantial recent theoretical progress in quantum algorithms for classical problems. For instance, algorithms have been developed for solving linear systems [1], simulating differential equations [2–5], machine learning [6–8], and various tasks in finance [9] and image processing [10–16].

However, it is often the case that simply *loading* classical data into the quantum computer is so costly that it overwhelms any quantum advantage that would have been expected in the execution of the main algorithm [17, 18]. In the general case, exact state preparation for a vector of length $M = 2^n$ into n qubits requires a circuit depth exponential in n [17, 19, 20] if ancilla qubits are not used. This "input problem" suggests two distinct research directions. First, it may be the case that some industrially relevant classical data, which is often quite structured, can in fact be loaded efficiently—hence studying existing classical data would bring practical insights and allow us to identify such data classes. Second, this problem motivates the development of algorithms that are more efficient for loading data, in order to mitigate this bottleneck.

In this work, we contribute to each of these research directions. First we introduce a novel tensor network (TN) algorithm called AMLET (Automatic Multi-layer Loader Exploiting TNs), demonstrating its improved performance relative to a modified version of a previously described TN algorithm for loading matrix product states from quantum problems. Second, we perform a broad numerical study to determine required circuit depths for loading various types of real-world classical data, including images, turbulent fluid data, financial data, and protein configurations.

There are several previous methods that attempt to address the input problem [19, 21–36]; we highlight some prominent approaches here. There are general methods with fixed circuit structures that scale exponentially with the number of qubits [19]; such methods are not easily amenable to tailorable space-depth tradeoffs. There are methods that can accurately load only some classes of data, for example smooth differentiable functions [27]. Others have proposed using neural network methods to generate quantum circuits for loading data [22]; such methods must be trained on the data class of interest and there does not appear to be a straightforward way to control error. Another notable strategy is to reduce depth at the cost of introducing additional ancilla qubits [30, 37]. Relatedly, quantum random-access memory (QRAM) [21] promises to partially mitigate the input problem, though such a solution would require more complex hardware and substantially more qubits. In this work, we show that AMLET produces accurate, optimization-free, ancilla-free, short-depth state preparation circuits for the amplitude encoding.

Algorithms for loading data. We begin with a high-level description of both circuit compilation algorithms. The first algorithm we implement is somewhat similar to the work of Ran [38], originally proposed not for classical data but for loading a matrix product state (MPS). We refer to it as the layer-by-layer algorithm. First one approximates the 2^n-length vector as an MPS, via repeatedly performing singular value decompositions (SVD) and truncations [39]. Then indices (*i.e.* edges) are added to the tensors on the edge of the tensor network

* nsawaya@protonmail.com

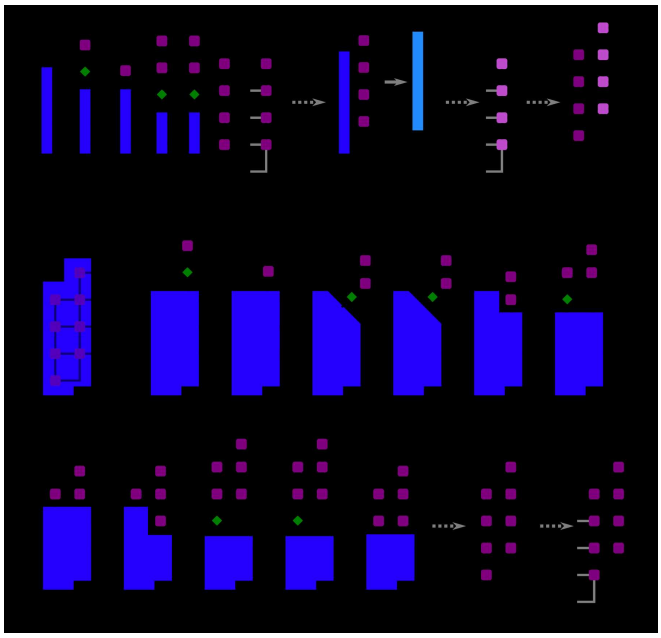

FIG. 1. Schematics of the 2-layer versions of the (a) layer-by-layer and (b) AMLET algorithms. Both algorithms may be generalized to an arbitrary number of layers $\mathcal{D}$. Green triangles denote the diagonal singular value tensor that results from performing SVD. '*' indicates that SVD truncation was performed, '†' that tensors were expanded to form unitary quantum gates, '‡' that the full circuit was contracted, and '§' that reindexing was performed. All singular value tensors (green) must be contracted "downwards," such that the unitary-like normalization condition is always conserved above. Notably, while the first approximation (SVD truncation) in layer-by-layer is at step *a.v*, two-layer AMLET does not require any approximation until step *b.xi*; more AMLET layers would result in delaying the first truncations even further and in fewer total truncations.

to form two-qubit gates (rank-four tensors), as shown schematically in Figure 1, where SO(4) symmetry is enforced by populating the new tensor values with the null space of the original lower-dimensional tensor. Note that a gate in SO(4) may be decomposed into an arbitrary gate set [40]. (Throughout this work we assume a gate-set of CNOTs and arbitrary one-qubit rotations.) One then contracts this "disentangling circuit" with the state vector, yielding a new 2^n-length vector. The full procedure is then repeated on the resulting vector, and the final quantum circuit is the concatenation of the resulting disentangling circuits.

Next we introduce our AMLET algorithm, which instead produces the entire quantum circuit "on the fly." SVD-and-truncation decompositions form the basis of this algorithm as well, but the important difference is that we split each new bond into two bonds each of size 2, while retaining the desired network structure. AMLET is more complex, requiring careful fusing, splitting, and reordering of indices, while keeping track of tensor positions.

Both methods allow for a monotonic trade off between accuracy and depth—if one needs to increase the accuracy of the loading circuit, one can do so simply by successively increasing the depth. Additionally, neither requires the optimization of circuit parameters (which can lead to pathologies such as barren plateaus [41]), as the constructed quantum circuit results directly from successive linear algebra steps in a deterministic number of operations. All algorithms were implemented using in-house code built on Scipy [42] and Quimb [43].

Schematic descriptions of the layer-by-layer and AMLET algorithms are shown in Figure 1. Each begins with a normalized 2^n-by-1 vector. As mentioned, the layer-by-layer algorithm is conceptually similar to previous work [33, 38] that used an SVD-and-truncation method to load a matrix product state (MPS) describing a physical system. The green diamonds are the diagonal singular value (SV) tensors, which result from SVD. For both algorithms, the green diamond is always pushed "downwards," which ensures that the "above" tensors retain the left-normalized form [44], i.e. the orthogonalization condition

$$\sum_{\sigma,i} U^{\dagger}_{i;i-1,\sigma} U_{i-1,\sigma;i'} = I_{i,i'} \tag{1}$$

where i and $i+1$ are respectively the indices above and below the tensor, I is the identity operator, and σ is one of the original indices. Another required routine is SVD truncation, whereby the middle index of the tensor decomposition is reduced to dimension K,

$$\sum_{a,b,c,d}^{l_a,l_b,l_c,l_d} U_{ab} S_{bc} V_{cd} \rightarrow \sum_{a,b,c,d}^{l_a,K,K,l_d} U_{ab} S_{bc} V_{cd} \tag{2}$$

where we have assumed singular value matrix S is ordered from largest to smallest value. For example in steps *a.v* and *b.xi* the tensors must be truncated to $K = 2$, to ensure that the eventual tensor network maps directly to rank-4 (*i.e.* two-qubit) tensors of appropriate size.

In steps *a.vii* and *b.xiv*, tensors in the network are expanded (*i.e.* indices are added) in order to create two-qubit gates. This tensor expansion results in gauge freedom, whereby additional tensor elements must be populated. Expanding a $2 \times 2 \times 2$ to a $2 \times 2 \times 2 \times 2$ tensor, and rearranging the elements to correspond to a 4×4 two-qubit quantum gate, we schematically describe this expansion via

$$\begin{pmatrix} * & * & \cdot & \cdot \\ * & * & \cdot & \cdot \\ * & * & \cdot & \cdot \\ * & * & \cdot & \cdot \end{pmatrix} \rightarrow \begin{pmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{pmatrix} \tag{3}$$

where $*$ denotes already-filled entries and $\cdot$ denotes undetermined entries (*i.e.* gauge freedom). Determining the empty columns [38] is equivalent to finding the null space of the filled columns, which may be done using a Gram-Schmidt type of procedure. Note that the bottom-most

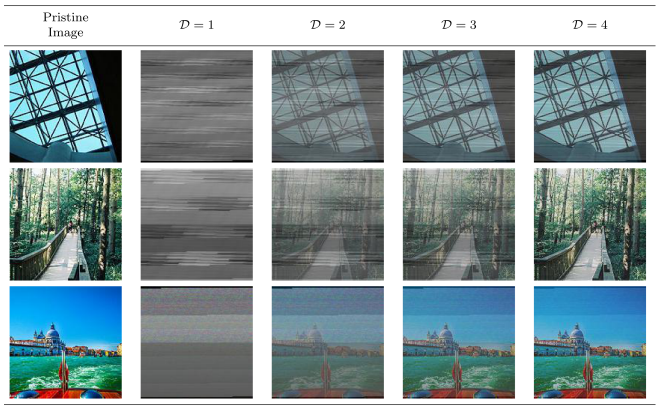

FIG. 2. Resulting approximate images from 14-qubit AMLET quantum circuits for $\mathcal{D} = 1$ through 4. In order the demonstrate that even the poorest results were relatively short-depth, these three images were chosen because they yielded the lowest fidelity in their image class.

tensors in Figure 1 instead require that three columns be filled. Finally, AMLET requires the additional subroutine of index splitting (reshaping) [45],

$$M_{ab} \to \tilde{M}_{acd}; \quad b = c \times d \tag{4}$$

which requires careful realignment of the tensor network to ensure correct ordering of tensor entries. This step is shown for example in *b.v*. Notably, the use of splitting in step *b.v* instead of truncating as in *a.v* ensures that approximations in AMLET are introduced significantly later, which in turns leads to fewer approximation (*i.e.* truncation) steps overall. By contrast, the layer-by-layer algorithm requires the introduction of approximations after almost every SVD operation. This is likely the reason for AMLET out-performing layer-by-layer in our results below.

The classical complexity of constructing the quantum circuit scales at most as $O(M)$ if the depth is bounded by a constant, by the following reasoning. Stated differently, forming a $\log_2(M)$-tensor MPS from a M-length vector scales at most as $O(M)$ if naive matrix subroutines are used. (More advanced subroutines *e.g.* the known $O(Q^{2.373})$-scaling matrix-matrix multiplication algorithm for $Q \times Q$ matrix can reduce this asymptotic scaling [46].) This is because naive SVD requires $O(ab^2)$ steps for an $a \times b$ matrix, where the first SVD-and-truncation is of a $2 \times \frac{M}{2}$ matrix ($ab^2 = 2M$), the second is of a $2 \times \frac{M}{4}$ matrix ($ab^2 = M$), and so on. In turn this leads to the sum $\sum_i^n \frac{M}{2^i} \sim O(M)$. When the number of layers is bounded by a constant a similar argument yields $O(M)$ for the AMLET algorithm, albeit with a different prefactor.

Classical datasets. Here we summarize the four types of classical data used in this work. Each uses a slightly different application-motivated quality measure. Note that in all cases the measure is based on the *unnormalized* vector, meaning that an increase in the vector norm of the original state does indeed lead to a more difficult loading problem. If we were to take the (flawed) approach of considering quality measures on normalized vectors, the circuit depths would be substantially shorter.

Images. We use the Intel Image Classification dataset [47], which contains six categories (buildings, forests, *etc.*) and was originally designed to benchmark classification algorithms [48]. We encode the two-dimensional RGB (red-green-blue) images in row-major format, where each color is grouped together (*i.e.* all red values are listed before all blue values). Note that researchers have proposed several other more complex formats [49] to represent images in quantum computers. The quality metric we use is

$$\|\vec{v}_{exact} - \vec{v}_{approx}\|_2 \leq \epsilon_I \tag{5}$$

where for images we set an arbitrary threshold $\epsilon_I < 10^{-3}$ where all vectors were length-2^{14}.

Fluid dynamics. Accurately simulating the Navier-Stokes equations in the turbulent regime is computationally expensive and often intractable for real-world problems [50], and there has been some effort in quantum algorithm development in this area [3]. We expect velocity data for laminar flow to be significantly easier to load than data from turbulent data, as the latter is less smooth. Hence we focus on (incompressible) turbulent velocity data (*i.e.* high Reynolds number) from a recently published dataset [51], where we limit ourselves to encoding only U_x, the velocity vector along the direction of the bulk flow. For this dataset, we ensure that the loaded data meets two error conditions. First, we bound ϵ_I by a constant. Second we ensure that total momentum is conserved in the incompressible fluid to within some error, which is achieved by ensuring that

$$|\sum(v_{exact}) - \sum(v_{approximate})| < \epsilon_M \tag{6}$$

where this expression may be derived by summing the velocities to produce momentum, and then bounding the error in momentum $|M_{exact} - M_{approx}|$. We estimate that choosing $\epsilon_M < 100$ yields a maximum relative error of at most 10^{-4} in the momentum of the entire block of fluid.

Finance. We use a dataset of minute-by-minute data for the following stock indices: S&P 500, NIKKEI 225, DAX 30, and EUROSTOXX [52]. We use equation (5) as a quality metric, with $\epsilon_I = 1$ currency unit (dollar, Euro, etc.). Notably, because the data is recorded only during trading hours, there are many sudden changes in value when two adjacent time points come from different days, an additional roughness that likely makes accurate loading more difficult.

Protein atomic positions. Proteins are a versatile and ubiquitous class of biomolecule, commonly studied by direct physics simulation [53] and also via statistical and machine learning models [54, 55]. Quantum algorithms have been proposed for porting such workloads to quantum computers [56, 57]. The metric we use is the atomic

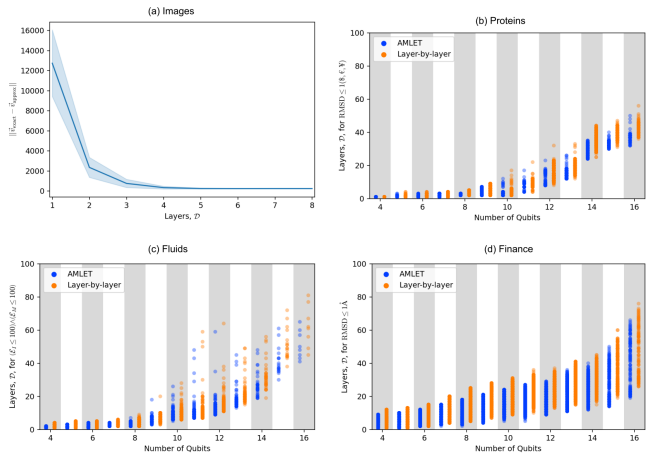

FIG. 3. Plot (a) shows the reduction in error in various images with increase in number of layers, using the AMLET algorithm; the plotted data is bounded by the maximum and minimum values over all images in the dataset. Plots (b), (c) and (d) show the number of layers needed to meet the given accuracy criteria for vectors representing positions of protein atomic coordinates, fluid velocity, and financial market indices respectively. The size of each classical vector is 2^n. Importantly, for all four classical application areas, the circuit depths are far shallower than the general amplitude encoding loading algorithm (a general amplitude encoding circuit would require the equivalent of at least $\mathcal{D} = 215$ layers for 12 qubits and of $\mathcal{D} = 2,715$ layers for 16 qubits, much larger than the vertical plot limits). Each of the application-specific error criteria are defined in the text.

root mean square deviation (RMSD),

$$\sqrt{\frac{1}{N_a}\sum_{i}^{N_a}\|\vec{r}_{j,exact} - \vec{r}_{j,approx}\|^2} < \epsilon_A, \tag{7}$$

where $\vec{r}_i$ is the three-coordinate position of atom j of N_a total atoms. This is proportional to equation (5) with an $1/M$ multiplicative factor. We use $\epsilon_I < 1$ Å, as in the vast majority of cases this is more precise than X-ray crystallography can achieve. We chose three of the largest proteins available in the Protein Data Bank [58]: protein entries 1VVJ [59], 3J3Q [60], and 3JA1 [61] are related to ribosome function, the HIV-1 capsid, and protein translation, respectively. Note that the data includes some atoms from RNA molecules. To obtain a larger dataset for each qubit count, we load non-overlapping sequences of atoms (hence the number of samples decreases as the qubit count increases).

Results & Discussion. We begin with the most important obervation from our results: though the depths are substantial in a subset of cases the depth does not increase exponentially, even though exactly loading the most general state indeed requires exponential depth [17, 19, 20]. A general amplitude encoding circuit [19, 40] would have required the equivalent of at least $\mathcal{D} = 215$ layers for 12 qubits and of $\mathcal{D}$ =2,715 layers for 16 qubits, while $\mathcal{D} < 90$ for AMLET (up to 16 qubits) with respect to all of our data sets.

The image dataset is the easiest to load of these data classes, likely because it is the most structured. Figure 2 visually demonstrates how quickly the features of the image become clear. After four layers, virtually all features are present. Note that the image dataset was the only set that was studied in an "intensive" as opposed to "extensive" way—in other words, we compressed each image to 14 qubits. During our tests we observed that the depth does not change substantially if we use a larger (less compressed) representation of the image, likely because there is little information being removed during compression.

For protein data, the number of layers increases smoothly, and a large subset of atomic coordinates requires just $\mathcal{D} < 40$ even for 20,000 atoms. Note that 16 qubits corresponds to $2^{16}/3 \approx 21,800$ atoms. Titin, the largest known protein, with ∼345,000 atoms, would require just $\log_2(3 \times 345,000) = 20$ qubits to load. Hence if the trend in Figure 3(b) persists then a quite conservative prediction is that even the largest known protein might be loadable in $\mathcal{D} < 200$.

The turbulent fluid data requires the highest circuit depths of the data we considered, though even 16-qubit vectors are loadable in a relatively few layers $\mathcal{D} < 90$. We attribute the anomolous jumps in maximum depth—*e.g.* the worst case 12-qubit circuit is deeper than the worst-case 14-qubit circuit—to error cancellation in the momentum, *i.e.* including more data points will often lead to a smaller results for formula (6).

Finally, the financial data shows a moderate increase in depth as qubit count increases, suggesting that financial time-series data is often loadable in manageable depths even up to tens of thousands of data points. Notably, the range (standard deviation) of depths is relatively small for each qubit count, suggesting that there is less variance in loading difficulty for financial data. This was not the case for the protein atomic positions, for which even some large vectors were loadable in very short depth.

In all cases, AMLET yields shorter depths than the layer-by-layer method for the same error. At 16 qubits, the reduction in is $\mathcal{D}$ between 8% and 25%. We attribute this to far fewer SVD truncations being required in the AMLET algorithm.

Conclusion. We have introduced AMLET, a tensor network-based compilation algorithm for loading a length-M classical vector into $\log_2(M)$ qubits (*i.e.* amplitude encoding), and used it to demonstrate that several classes of real-world classical data can be loaded in short depth. Further, we demonstrated AMLET's superior performance over a "layer-by-layer" algorithm that was inspired by previous work. The methods we introduced are depth-tunable, *i.e.* one can trivially achieve a trade-off between accuracy and depth. The benefits of AMLET include: (a) depth-accuracy tunability, (b) absence of any optimization parameters (eliminating concerns about optimization time and/or barren plateaus), (c) absence of ancilla qubits, and (d) shorter-depth re-

sults than the layer-by-layer approach. Our numerical studies showed data being loaded in orders of magnitude lower depth than the general loading algorithm would yield. This study suggests paths to valuable future work: for instance, one could apply AMLET to encodings more complex than the standard amplitude encoding; another possibility is a modified version of AMLET that adheres to an arbitrary circuit pattern or hardware topology. In conclusion, besides introducing a new algorithm that will be useful in quantum circuit compilation, this work demonstrates that the "input problem" will likely *not* be an obstacle to quantum advantage for many scientifically interesting classical workloads.

ACKNOWLEDGEMENTS

We are grateful to Shavindra Premaratne, Roza Kotlyar, and Anne Matsuura for useful discussions, and we thank Daan Camps for providing valuable feedback on the manuscript.

[1] A. W. Harrow, A. Hassidim, and S. Lloyd, Quantum algorithm for linear systems of equations, Physical Review Letters **103**, 150502 (2009).
[2] D. W. Berry, High-order quantum algorithm for solving linear differential equations, Journal of Physics A: Mathematical and Theoretical **47**, 105301 (2014).
[3] S. Lloyd, G. De Palma, C. Gokler, B. Kiani, Z.-W. Liu, M. Marvian, F. Tennie, and T. Palmer, Quantum algorithm for nonlinear differential equations, arXiv preprint arXiv:2011.06571 (2020).
[4] M. Lubasch, J. Joo, P. Moinier, M. Kiffner, and D. Jaksch, Variational quantum algorithms for nonlinear problems, Physical Review A **101**, 010301 (2020).
[5] A. M. Childs, J.-P. Liu, and A. Ostrander, High-precision quantum algorithms for partial differential equations, Quantum **5**, 574 (2021).
[6] A. Perdomo-Ortiz, M. Benedetti, J. Realpe-Gómez, and R. Biswas, Opportunities and challenges for quantum-assisted machine learning in near-term quantum computers, Quantum Science and Technology **3**, 030502 (2018).
[7] M. Schuld and N. Killoran, Quantum machine learning in feature Hilbert spaces, Physical Review Letters **122**, 040504 (2019).
[8] M. Cerezo, G. Verdon, H.-Y. Huang, L. Cincio, and P. J. Coles, Challenges and opportunities in quantum machine learning, Nature Computational Science **2**, 567 (2022).
[9] D. Herman, C. Googin, X. Liu, Y. Sun, A. Galda, I. Safro, M. Pistoia, and Y. Alexeev, Quantum computing for finance, Nature Reviews Physics , 1 (2023).
[10] Y. Zhang, K. Lu, and Y. Gao, QSobel: a novel quantum image edge extraction algorithm, Science China Information Sciences **58**, 1 (2015).
[11] S. Jiang, R.-G. Zhou, W. Hu, and Y. Li, Improved quantum image median filtering in the spatial domain, International Journal of Theoretical Physics **58**, 2115 (2019).
[12] N. Jiang, J. Wang, and Y. Mu, Quantum image scaling up based on nearest-neighbor interpolation with integer scaling ratio, Quantum Information Processing **14**, 4001 (2015).
[13] H.-S. Li, P. Fan, H.-y. Xia, S. Song, and X. He, The multi-level and multi-dimensional quantum wavelet packet transforms, Scientific Reports **8**, 1 (2018).
[14] S. Caraiman and V. I. Manta, Quantum image filtering in the frequency domain, Advances in Electrical and Computer Engineering **13**, 77 (2013).
[15] S. Caraiman and V. I. Manta, Histogram-based segmentation of quantum images, Theoretical Computer Science **529**, 46 (2014).
[16] K. Nakaji and N. Yamamoto, Quantum semi-supervised generative adversarial network for enhanced data classification, Scientific Reports **11**, 1 (2021).
[17] S. Aaronson, Read the fine print, Nature Physics **11**, 291 (2015).
[18] E. Tang, Quantum principal component analysis only achieves an exponential speedup because of its state preparation assumptions, Phys. Rev. Lett. **127**, 060503 (2021).
[19] M. Plesch and Č. Brukner, Quantum-state preparation with universal gate decompositions, Physical Review A **83**, 032302 (2011).
[20] D. Poulin, A. Qarry, R. Somma, and F. Verstraete, Quantum simulation of time-dependent Hamiltonians and the convenient illusion of hilbert space, Physical Review Letters **106**, 170501 (2011).
[21] V. Giovannetti, S. Lloyd, and L. Maccone, Quantum random access memory, Physical Review Letters **100**, 160501 (2008).
[22] C. Zoufal, A. Lucchi, and S. Woerner, Quantum generative adversarial networks for learning and loading random distributions, npj Quantum Information **5**, 1 (2019).
[23] I. F. Araujo, D. K. Park, F. Petruccione, and A. J. da Silva, A divide-and-conquer algorithm for quantum state preparation, Scientific Reports **11**, 1 (2021).
[24] K. Korzekwa, Z. Puchała, M. Tomamichel, and K. Życzkowski, Encoding classical information into quantum resources, IEEE Transactions on Information Theory (2022).
[25] J. Bausch, Fast black-box quantum state preparation, Quantum **6**, 773 (2022).
[26] R. LaRose and B. Coyle, Robust data encodings for quantum classifiers, Physical Review A **102**, 032420 (2020).
[27] A. Holmes and A. Matsuura, Efficient quantum circuits for accurate state preparation of smooth, differentiable functions, in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, 2020) pp. 169–179.
[28] G. Marin-Sanchez, J. Gonzalez-Conde, and M. Sanz, Quantum algorithms for approximate function loading, arXiv preprint arXiv:2111.07933 (2021).
[29] T. Shirakawa, H. Ueda, and S. Yunoki, Automatic quantum circuit encoding of a given arbitrary quantum state, arXiv preprint arXiv:2112.14524 (2021).
[30] X.-M. Zhang, M.-H. Yung, and X. Yuan, Low-depth quantum state preparation, Phys. Rev. Res. **3**, 043200

(2021).
[31] M. G. Amankwah, D. Camps, E. Bethel, R. Van Beeumen, and T. Perciano, Quantum pixel representations and compression for N-dimensional images, Scientific Reports **12**, 1 (2022).
[32] D. Camps and R. Van Beeumen, FABLE: Fast approximate quantum circuits for block-encodings, arXiv preprint arXiv:2205.00081 (2022).
[33] M. S. Rudolph, J. Chen, J. Miller, A. Acharya, and A. Perdomo-Ortiz, Decomposition of matrix product states into shallow quantum circuits, arXiv preprint arXiv:2209.00595 (2022).
[34] M. T. West, A. C. Nakhl, J. Heredge, F. M. Creevey, L. C. Hollenberg, M. Sevior, and M. Usman, Drastic circuit depth reductions with preserved adversarial robustness by approximate encoding for quantum machine learning, arXiv preprint arXiv:2309.09424 (2023).
[35] A. A. Melnikov, A. A. Termanova, S. V. Dolgov, F. Neukart, and M. Perelshtein, Quantum state preparation using tensor networks, Quantum Science and Technology (2023).
[36] I. Y. Akhalwaya, A. Connolly, R. Guichard, S. Herbert, C. Kargi, A. Krajenbrink, M. Lubasch, C. M. Keever, J. Sorci, M. Spranger, *et al.*, A modular engine for quantum Monte Carlo integration, arXiv preprint arXiv:2308.06081 (2023).
[37] K. Gui, A. M. Dalzell, A. Achille, M. Suchara, and F. T. Chong, Spacetime-efficient low-depth quantum state preparation with applications, arXiv preprint arXiv:2303.02131 (2023).
[38] S.-J. Ran, Encoding of matrix product states into quantum circuits of one-and two-qubit gates, Physical Review A **101**, 032310 (2020).
[39] R. Orús, A practical introduction to tensor networks: Matrix product states and projected entangled pair states, Annals of physics **349**, 117 (2014).
[40] F. Vatan and C. Williams, Optimal quantum circuits for general two-qubit gates, Physical Review A **69**, 032315 (2004).
[41] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, Barren plateaus in quantum neural network training landscapes, Nature communications **9**, 4812 (2018).
[42] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, *et al.*, Scipy 1.0: fundamental algorithms for scientific computing in python, Nature methods **17**, 261 (2020).
[43] J. Gray, quimb: A python package for quantum information and many-body calculations, Journal of Open Source Software **3**, 819 (2018).
[44] U. Schollwöck, The density-matrix renormalization group in the age of matrix product states, Annals of physics **326**, 96 (2011).
[45] S. Singh, R. N. Pfeifer, and G. Vidal, Tensor network states and algorithms in the presence of a global U(1) symmetry, Physical Review B **83**, 115125 (2011).
[46] P. Bürgisser, M. Clausen, and M. A. Shokrollahi, *Algebraic complexity theory*, Vol. 315 (Springer Science & Business Media, 2013).
[47] Intel image classification (Kaggle dataset) (2018), available at `kaggle.com/datasets/puneet6060/intel-image-classification`.
[48] X. Wu, R. Liu, H. Yang, and Z. Chen, An xception based convolutional neural network for scene image classification with transfer learning, in *2020 2nd International Conference on Information Technology and Computer Application (ITCA)* (IEEE, 2020) pp. 262–267.
[49] F. Yan, A. M. Iliyasu, and S. E. Venegas-Andraca, A survey of quantum image representations, Quantum Information Processing **15**, 1 (2016).
[50] W. Rodi, Turbulence modeling and simulation in hydraulics: A historical review, Journal of Hydraulic Engineering **143**, 03117001 (2017).
[51] R. McConkey, E. Yee, and F.-S. Lien, A curated dataset for data-driven turbulence modelling, Scientific data **8**, 1 (2021).
[52] Huge stock price data: Intraday minute bar (Kaggle dataset) (2020), available at `kaggle.com/datasets/arashnic/stock-data-intraday-minute-bar`.
[53] P. Śledź and A. Caflisch, Protein structure-based drug design: from docking to molecular dynamics, Current opinion in structural biology **48**, 93 (2018).
[54] M. Baek, F. DiMaio, I. Anishchenko, J. Dauparas, S. Ovchinnikov, G. R. Lee, J. Wang, Q. Cong, L. N. Kinch, R. D. Schaeffer, *et al.*, Accurate prediction of protein structures and interactions using a three-track neural network, Science **373**, 871 (2021).
[55] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, *et al.*, Highly accurate protein structure prediction with AlphaFold, Nature **596**, 583 (2021).
[56] A. Robert, P. K. Barkoutsos, S. Woerner, and I. Tavernelli, Resource-efficient quantum algorithm for protein folding, npj Quantum Information **7**, 1 (2021).
[57] P. A. M. Casares, R. Campos, and M. A. Martin-Delgado, QFold: quantum walks and deep learning to solve protein folding, Quantum Science and Technology **7**, 025013 (2022).
[58] S. K. Burley, C. Bhikadiya, C. Bi, S. Bittrich, L. Chen, G. V. Crichlow, C. H. Christie, K. Dalenberg, L. Di Costanzo, J. M. Duarte, *et al.*, RCSB Protein Data Bank: powerful new tools for exploring 3D structures of biological macromolecules for basic and applied research and education in fundamental biology, biomedicine, biotechnology, bioengineering and energy sciences, Nucleic acids research **49**, D437 (2021).
[59] T. Maehigashi, J. A. Dunkle, S. J. Miles, and C. M. Dunham, Structural insights into +1 frameshifting promoted by expanded or modification-deficient anticodon stem loops, Proceedings of the National Academy of Sciences **111**, 12740 (2014).
[60] G. Zhao, J. R. Perilla, E. L. Yufenyuy, X. Meng, B. Chen, J. Ning, J. Ahn, A. M. Gronenborn, K. Schulten, C. Aiken, *et al.*, Mature HIV-1 capsid structure by cryo-electron microscopy and all-atom molecular dynamics, Nature **497**, 643 (2013).
[61] W. Li, Z. Liu, R. K. Koripella, R. Langlois, S. Sanyal, and J. Frank, Activation of GTP hydrolysis in mRNA-tRNA translocation by elongation factor G, Science advances **1**, e1500169 (2015).